Компиляторы. Избавление от вложенных функций

Косарев Дмитрий

6 сентября 2025 г.

Дата сборки: 5 сентября 2025 г.



Вложенные функции и замыкания

```
let fac n =
  let rec fack n k =
    if n <= 1 then k 1
    else fack (n-1) (fun m -> k (m * n))
  in
  fack n (fun x -> x)
```

В OCaml используются вложенные функции и анонимные функции, которых в явном виде нет ни в Си, ни в процессоре. Чтобы OCaml скомпилировать, надо сделать что-то с этими конструкциями и замыканиями

Closure conversion

Решение прямолинейное: найти все имена, которые «замыкаются» и передать их (почти) явно. Тут есть несколько вариантов

- Передать явно (разберем подробнее)
- Сложить в n-ку
- Связи доступа (англ. chain links) и дисплеи ([**dragon**] глава 7.3)

Поиск таких имен:

- Идем по дереву абстрактного синтаксиса (AST)
- Имя встречается — добавляем
- λ-абстракция – удаляем имя
- В конце удаляем имена из глобальной области видимости (stdlib, например)

Затем надо найденные имена передать явно

```
let fac n =
  let rec fack n k =
    if n<=1 then k 1
    else fack (n-1) ((fun k n m -> k (m * n)) k n)
  in
  fack n (fun x -> x)
```



```
let fac n =
  let fack1 k n m = k (m * n) in
  let rec fack n k =
    if n<=1 then k 1
    else fack (n-1) (fack1 k n)
  in
  fack n (fun x -> x)
```

А затем — lambda lifting

```
let fac n =
  let fack1 k n m = k (m * n) in
  let rec fack n k =
    if n<=1 then k 1
    else fack (n-1) (fack1 k n)
  in
  fack n (fun x -> x)
```

↴

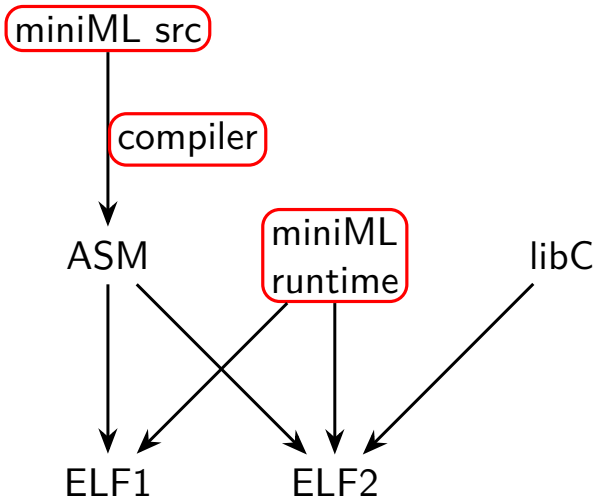
```
let fack1 k n m = k (m * n)

let rec fack n k =
  if n<=1 then k 1
  else fack (n-1) (fack1 k n)

let fac n = fack n (fun x -> x)
```

В коде осталось одно частичное применение

Картинка с libc и линкером



Рантайм включает в себя

- Создание замыкания
- Вызов замыкания
- Вызов IO (для дебага)
- Создание данных (n-ки)
- GC
- и т.д.

N.B.

- В компиляторе должны уметь вызываться функции произвольной арности
- Вызываться эффективно!

Ссылки & Acknowledgements


Mattew Might. *Closure conversion: How to compile lambda*. url: https://matt.might.net/articles/closure-conversion.


Alfred V. Aho и др. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. 2006.


Thomas Johnsson. «Lambda lifting: Transforming programs to recursive equations». В: 1985.


Xavier Leroy. *Part V: functional intermediate representations*. url: https://xavierleroy.org/mpri/2-4/fir.pdf.